

Technical Analysis Advisor

Live Intraday Signal Generation for Liquid Market Instruments

By- Anubhav Singh ; 22515500012 ; NE

Project Synopsis · B.Tech CSE (AIML), 2nd Year

Machine Learning · Data Visualization · Statistics & Probability · Software Engineering

1. The Problem

Retail traders who follow liquid instruments during the trading day usually switch between a broker app and a separate charting tool, manually re-checking indicators such as RSI and moving-average crossovers every few minutes. By the time the whole watchlist has been reviewed, short-lived intraday opportunities have often already passed.

This project builds a live dashboard the trader keeps open during market hours. It pulls near-real-time price data for a small watchlist, computes technical indicators on each new price update, and shows a continuously refreshing buy/sell-lean signal with a confidence score for each instrument. The exact layout of this information is left to the student's design.

Traders or Investors give a lot of time reviewing multiple strategies, candle stick patterns and indicators such as RSI, MACD and EMA. This tool will help them to combine all + integrate Machine learning to predict near future movements from actual price actions.

Scope note: this is an academic decision-support prototype, not a trading bot. No real orders are ever placed. The interface must show a visible disclaimer: "For educational purposes only — not investment advice."

2. Which Instrument, and Where the Data Comes From

The project isn't tied to any one market. Any liquid, actively traded instrument works, as long as its data is free and doesn't need a login. The recommended default is **BTC-USD (Bitcoin)**, pulled through Yahoo Finance — crypto trades 24/7, which avoids the extra complexity of scheduling around market open/close hours for a first version. Equally valid alternatives: a large, high-volume stock (AAPL, RELIANCE.NS), an index ETF (SPY), or a major forex pair (EURUSD=X). The pipeline design below stays identical no matter which one is chosen.

Data sources considered:

- **yfinance (primary choice)** — completely free, no signup or key needed. Gives 1-minute and 5-minute price bars, covering roughly the last 60 days. It's an unofficial API and can occasionally rate-limit, which should be noted as a risk in the SRS.
- **Alpha Vantage / Twelve Data (fallback only)** — also free, but requires a free API key signup. Use only if yfinance is unavailable in the deployment environment.
- **Kite optional.**

For training the model (a one-time historical pull, separate from the live feed): yfinance's own 60-day, 5-minute history gives roughly 17,000 rows for a 24/7 instrument like BTC-USD — comfortably within the required data-size range. One limitation worth stating plainly in the SRS: Yahoo Finance caps intraday history at about 60 days, so years of 5-minute data simply aren't available this way.

3. What the Model Predicts, and How It's Judged

Task: classify the direction of the next 5-minute candle — up or down — using indicators computed from the current candle (RSI, MACD, rolling volatility, volume change). This is a single-snapshot prediction, not a sequence-forecasting model.

Metric: precision@K, where K is the number of watchlist instruments the trader actually watches (for example, K = 5). In plain terms: out of the K signals shown, how many turned out to be correct.

Two baselines the model must beat to mean anything:

- **Floor baseline:** always guess “up,” based on the instrument's historical tendency.
- **No-ML baseline:** guess the majority direction for that time of day (e.g. the opening 15 minutes behaves differently from the midday lull) — this is the single most obvious pattern at this timescale.

All three numbers — floor, no-ML baseline, and the actual model — get reported together, along with the percentage improvement over the no-ML baseline. Five-minute price direction is close to random, so if the model struggles to beat the baseline, that's a legitimate and worth-reporting finding, not a failure.

4. Techniques, Statistics & Visualization

Modelling approach (suggestions — a substitute is fine if justified in the SRS):

- Logistic regression as the interpretable baseline model
- Decision tree, for a rules-style explanation of each signal
- AdaBoost or bagging, to lift performance beyond the plain baseline
- SVM as a comparison model
- Most of the real work is in feature engineering — building RSI, MACD, moving-average crossovers and rolling volatility — not in choosing the classifier.

Statistics that make the model trustworthy, not just decoration:

- Base rate of up-candles, broken down by time of day
- A confidence interval around the precision@K score, recalculated daily as more live data comes in
- Correlation between rolling volatility and how often the signal turns out wrong

What the user sees (React dashboard):

- A live watchlist with a signal badge per instrument
- An auto-updating chart showing price and the indicators behind each signal
- A running “how often has this been right” score per instrument

What the analyst sees (Power BI / Tableau report):

- A heatmap of signal accuracy by time of day
- A KPI card showing overall precision@K, updated daily
- Drill-down views by instrument and by week

5. How It's Built (MERN + a Python ML Service)

Since Node doesn't have a mature equivalent to scikit-learn, the machine-learning piece runs as its own small Python service that the rest of the app talks to over a simple API call.

- **Ingestion (Python, scheduled):** checks yfinance every 5 minutes during the instrument's active session, computes the indicator features for each new candle, and saves them to the database.
- **ML service (Flask):** holds the model, trained once offline on the historical data. Given the latest feature values, it returns a signal and a confidence score. It is not retrained live in this version.
- **Backend (Express):** ties everything together and pushes fresh predictions to the dashboard using Socket.io, so the page updates on its own without the user hitting refresh.
- **Frontend (React):** the live dashboard itself — watchlist, chart, signal badges, and the required disclaimer banner.
- **Database (MongoDB):** stores incoming price data, computed features, past predictions, and the running accuracy score per instrument.

6. Time Budget (90 Hours Total)

Planning, SRS, and confirming data access	15 hrs
Building the data pipeline and engineering features	15 hrs
Training and evaluating the model against baselines	20 hrs
Building the live data pipeline and real-time updates	18 hrs
Building the dashboard and the analyst report	14 hrs
Testing, final report, and demo preparation	8 hrs

If the live pipeline turns out to be too heavy: refresh every 15 minutes instead of 5, and use simple polling from React instead of Socket.io. It still feels live to the user, while removing the riskiest piece of infrastructure from the critical path.

Prototype for academic evaluation only. Not investment advice. No live order execution is implemented.
*****There may be necessary changes to be executed in future on this documentation*****